

허니팟 기반 사이버 공격 분석 시스템

공격자를 유인해 기록하는 것까지는 오픈소스가 해준다. 그 영문 로그를 **비전문 관리자가 그대로 읽고 바로 조치할 수 있는 한국어 위협 보고**로 자동 변환하는 4단 파이프라인을 만들었다.

기반 T-Pot 24.04.1 구성 컨테이너 10 · 사이드카 4 코드 약 7,800줄 기준일 2026-09-09

허니팟이란, 그리고 무엇이 비어 있었다

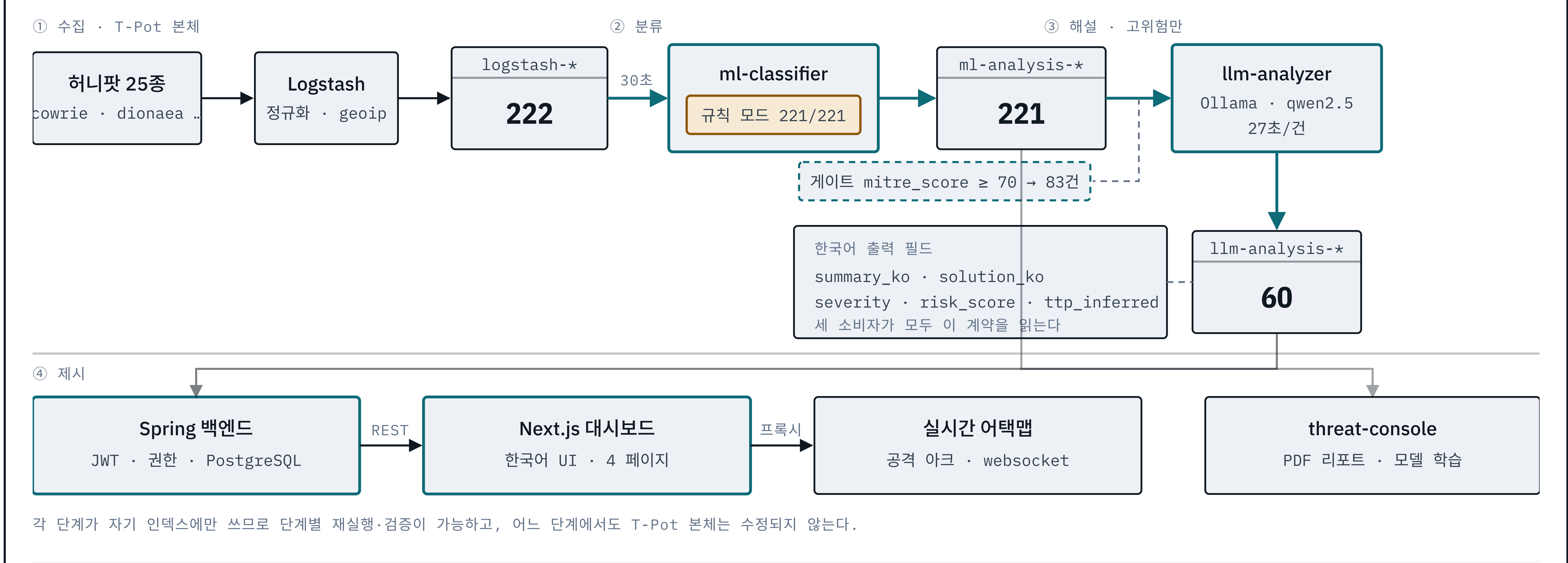
허니팟은 일부러 취약하게 열어둔 미끼 시스템이다. 실제 서비스가 아니므로 여기 들어오는 접속은 정의상 전부 비정상이고, 그래서 **오탐이 거의 없는 공격 정보**를 얻는다. 공격자가 가짜 셸에 `wget` 으로 악성코드를 내려받으면 **그 명령이 그대로 기록된다** — 실제 시스템은 안전한 채로.

T-Pot 은 허니팟 25종을 묶어 로그를 쌓아준다. 수집은 이미 해결돼 있다. 그러나 남는 것은 `cowrie.login.failed` 같은 영문 원시 이벤트뿐이고, 이를 "무슨 일이 있었고 뭘 해야 하나"로 옮기려면 허니팟별 스키마와 MITRE 를 이미 알아야 한다.

목표는 **그 해석을 자동화하는 것이다**. 모든 이벤트에 공격 유형과 위협 점수를 매기고, 고위험 건은 한국어 요약과 실행 가능한 대응 방안까지 만들어 대시보드에 올린다. 전담 보안 인력이 없는 조직도 자기 대역을 노리는 공격을 한국어로 통보받을 수 있다.

수집 이벤트 222 logstash-*	분류 완료 221 고위험 83
한국어 해설 60 폴백 0건	분류 모드 규칙 근거는 아래

시스템 구조 — 사이드카 4종, T-Pot 본체는 수정하지 않는다



사이드카 방식. 허니팟·Logstash·Kibana 를 건드리지 않고 ES 를 읽어 자기 인덱스에만 쓰는 컨테이너를 덧붙였다. 업스트림 업데이트와 충돌하지 않고, 실패해도 이전 단계는 그대로 남는다.

ML 을 켜지 않은 이유

학습 모델의 holdout 정확도는 **0.998**이었다. 그런데 실제 문서를 넣으니 60건이 **전부 Recon** 으로 나왔다. 네 차례 실험으로 원인을 좁혔다.

① 원본	35.4%
② 라벨 재균형	35.4%
③ 세션 조인	61.4%
④ payload 추가	61.4%

규칙 대비 일치율(실제 ES 문서 127건). 세션 조인으로 `dst_port` 결손 48%→100%, `login_attempts` 0%→93% 복원 — **26pp 상승**.

천장의 정체
라벨이 규칙에서 파생되므로 모델의 최대치는 규칙 재현(100%) 이다. 게다가 payload 로 얻은 리버스셸 384행 중 322행의 라벨이 Recon 이었다 — 특징이 생겨도 라벨이 그 특징을 부정한다.

결론: 61.4% 로 켜면 운영이 나빠진다. 규칙 모드를 유지한다.

LLM 에게 맡길 일과 맡기지 않을 일

`mitre_score` 92 인 멀웨어 문서에 모델이 `severity: LOW` 를 반환했다. 파이프라인이 이미 70 이상만 통과시키므로 **내부 모순**이다. **고치기 전에 잤다** — 프롬프트 문제와 능력 문제는 해법이 다르다.

기본 프롬프트	10 / 10 실패
판정 예시 3개 추가	10 / 10 실패

few-shot 을 넣어도 **전혀 바뀌지 않았다** — 프롬프트가 아니라 3B 모델의 능력 한계다.

설계 원칙
결정론적으로 계산 가능한 값은 LLM 에게 묻지 않는다. 위험도는 점수에서 유도하고, 모델은 요약·대응방안·기법추정처럼 언어 능력이 필요한 일만 맡는다.

더 강한 모델엔 `LLM_SEVERITY=model` 로 판정을 되돌려준다. 그때도 **모델은 올릴 수는 있어도 내릴 수 없다**.

검증과 다음 단계

파이프라인 4단	● 동작
한국어 해설	60 / 60 ok
폴백 · 한자 혼입	0 · 0
어택맵 websocket	101 · 12프레임
백엔드 · 프론트	6종 · 4페이지 200

조용한 실패 4건

모두 **예외도 경고도 없이** 잘못된 값이나 빈 화면을 냈다.

- `int("23.0")` 예외 → 포트가 전부 0
- Suricata `payload` 미조회 → 명령 특징 소실
- ES 동적 매핑 → 집계가 빈 차트로 삼켜짐
- 한국어에 한자 혼입 → 출력 검사로 차단

개선 방향

- 실 데이터 학습** — ES 문서로 직접 학습하면 skew 자체가 사라진다
- 모델 상향** — 8B/외부 API 전환 시 `severity_source` 분포가 곧 측정치
- 규칙 라벨러 개선** — 내용 기반 분기를 타 허니팟으로 확대